

BÁO CÁO KỸ THUẬT: CHIẾN LƯỢC TRIỂN KHAI HỆ THỐNG MINI-ERP VÀ THƯƠNG MẠI ĐIỆN TỬ CHO THỜI TRANG TRUYỀN THỐNG (OFFLINE-FIRST)

Tóm tắt điều hành

Báo cáo này trình bày một kế hoạch kỹ thuật toàn diện và lộ trình triển khai kéo dài 30 tuần nhằm xây dựng một hệ sinh thái quản trị nguồn lực doanh nghiệp thu nhỏ (Mini-ERP) chuyên biệt cho ngành thời trang truyền thống Việt Nam, cụ thể là Áo dài và Áo bà ba. Dự án được thiết kế để giải quyết các thách thức đặc thù của ngành bán lẻ thủ công, bao gồm sự phức tạp trong quản lý kích thước (từ size S đến 8X), sự đa dạng của chất liệu gấm (hoa mai, chữ Thọ, trúc đào), và mô hình kinh doanh phi tập trung kết nối với mạng lưới thợ sửa đồ.¹

Điểm cốt lõi của giải pháp là kiến trúc **"Offline-First"** (Ưu tiên ngoại tuyến), cho phép ứng dụng vận hành tự chủ trên các thiết bị khách (điện thoại, máy tính) thông qua cơ sở dữ liệu cục bộ SQLite, đồng thời duy trì tính toàn vẹn dữ liệu với máy chủ trung tâm MySQL thông qua các thuật toán đồng bộ hóa phức tạp.¹ Hệ thống tích hợp sâu công nghệ **Thị giác máy tính (Computer Vision)** sử dụng mô hình YOLOv8 thông qua giao diện Deep Java Library (DJL) để tự động hóa quy trình nhập kho, phân tích kết cấu vải và dự đoán niên đại trang phục dựa trên dữ liệu thời gian thực.¹

Lộ trình phát triển được tối ưu hóa cho môi trường phát triển **NetBeans IDE**, tận dụng các công cụ tạo mã tự động và quản lý dự án Maven của IDE này để đảm bảo tiến độ và chất lượng mã nguồn Java Spring Boot.

Phần 1: Kiến trúc Hệ thống và Chiến lược Dữ liệu (Cơ sở lý luận)

Trước khi đi vào chi tiết triển khai từng bước, việc thiết lập một nền tảng kiến trúc vững chắc là điều kiện tiên quyết để đảm bảo hệ thống có thể xử lý các yêu cầu phức tạp về đồng bộ hóa và xử lý ảnh.

1.1. Kiến trúc Offline-First và Thách thức Đồng bộ hóa

Trong môi trường bán lẻ truyền thống và kho bãi, kết nối mạng thường không ổn định. Kiến trúc Offline-First đảo ngược mô hình client-server truyền thống: ứng dụng coi cơ sở dữ liệu

cục bộ là nguồn chân lý (source of truth) cho các tương tác của người dùng, và việc đồng bộ với máy chủ chỉ diễn ra dưới nền.¹

Cơ chế Định danh Phân tán (Distributed Identity Strategy)

Một trong những thách thức lớn nhất khi xây dựng hệ thống phân tán cho phép ghi dữ liệu ngoại tuyến là xung đột khóa chính (Primary Key). Trong các hệ thống MySQL truyền thống, khóa chính thường là số nguyên tự tăng (AUTO_INCREMENT). Tuy nhiên, nếu hai thiết bị ngoại tuyến cùng tạo một đơn hàng mới với ID là 100, khi đồng bộ lên máy chủ sẽ xảy ra xung đột dữ liệu nghiêm trọng.²

Giải pháp được đề xuất và áp dụng trong dự án này là sử dụng **UUID (Universally Unique Identifier)** chuẩn 128-bit làm khóa chính cho tất cả các bảng dữ liệu quan trọng (Sản phẩm, Đơn hàng, Phiên sửa chữa).

- **Cơ chế:** Khi thiết bị khách (Client) tạo một bản ghi mới, nó tự sinh ra một chuỗi UUID (ví dụ: 550e8400-e29b-41d4-a716-446655440000) bằng thư viện java.util.UUID của Java. Xác suất trùng lặp của UUID là gần như bằng không, cho phép các thiết bị hoạt động hoàn toàn độc lập mà không cần "xin" ID từ máy chủ.⁴
- **Tác động hiệu năng:** Mặc dù UUID chiếm 16 byte so với 4 byte của số nguyên, và có thể gây phân mảnh chỉ mục trong MySQL (do tính ngẫu nhiên), nhưng sự an toàn về dữ liệu trong môi trường offline là ưu tiên cao hơn. Để giảm thiểu tác động này, chúng ta sẽ cấu hình MySQL sử dụng ORDER BY tối ưu và cân nhắc sử dụng UUID phiên bản 7 (có sắp xếp theo thời gian) nếu khả thi.⁵

Ma trận Giải quyết Xung đột (Conflict Resolution Matrix)

Khi dữ liệu được đồng bộ, xung đột là không thể tránh khỏi (ví dụ: Admin sửa giá trên server trong khi nhân viên kho sửa mô tả trên điện thoại). Hệ thống sẽ áp dụng ma trận giải quyết sau:

Loại Dữ liệu	Hướng Đồng bộ	Chiến lược Xung đột	Cơ sở Lý luận
Danh mục Sản phẩm	Server \$\\rightarrow\$ Client	Server Wins	Dữ liệu gốc (Master Data) do Admin quản lý tập trung phải là chuẩn mực cuối cùng.
Đơn hàng (Orders)	Client \$\\rightarrow\$	Append Only	Đơn hàng là các sự kiện giao dịch bất

	Server		biến. Không sửa đơn hàng cũ, chỉ thêm mới hoặc tạo đơn điều chỉnh.
Tồn kho (Inventory)	Hai chiều (Bi-directional)	Delta Logic	Thay vì ghi đè số lượng (ví dụ: "Kho = 5"), client gửi thay đổi (ví dụ: "Bán -1"). Server tính toán: Mới = Cũ + Delta. Điều này ngăn chặn việc bán âm kho khi nhiều thiết bị bán cùng lúc. ⁷
Hồ sơ Khách hàng	Hai chiều	Last Write Wins (LWW)	Dựa trên timestamp, thay đổi mới nhất sẽ được ghi nhận (ví dụ: khách hàng cập nhật số đo).

1.2. Mô hình Dữ liệu Quan hệ (MySQL Schema Design)

Sơ đồ cơ sở dữ liệu phải phản ánh sự phức tạp của sản phẩm thời trang.

- **Bảng products:** Lưu trữ thông tin chung.
 - id (CHAR(36), PK): UUID.
 - design_year_prediction (INT): Lưu trữ kết quả dự đoán từ AI về niên đại kiểu dáng.
 - fabric_class (VARCHAR): Lưu trữ phân loại vải từ AI (Gấm, Lụa, Nhung).
- **Bảng product_variants:** Quản lý biến thể SKU.
 - size (VARCHAR): Hỗ trợ dải size đặc biệt (S, M, L... 5X, 6X, 7X, 8X).¹
 - color_hex (VARCHAR): Mã màu chủ đạo trích xuất từ ảnh.
 - import_price (DECIMAL): Giá nhập (Được mã hóa hoặc phân quyền chặt chẽ).
 - retail_price (DECIMAL): Giá bán lẻ.
- **Bảng inventory_ledger:** Sổ cái kho hàng.
 - Thay vì chỉ lưu số lượng hiện tại, bảng này lưu lịch sử giao dịch: variant_id, change_amount (+10 nhập, -1 bán), transaction_type, created_at. Điều này hỗ trợ truy vết và báo cáo chính xác.
- **Bảng repair_ecosystem:** Quản lý thợ sửa.
 - tailor_id (FK), customer_id (FK), fee_amount (100% doanh thu thuộc về thợ), rating (1-5 sao).

Phần 2: Kế hoạch Triển khai Chi tiết 30 Tuần (Giai đoạn & Công cụ)

Kế hoạch được chia thành 6 giai đoạn chính, mỗi giai đoạn kéo dài từ 4-6 tuần, tập trung vào việc hoàn thiện từng nhóm chức năng cốt lõi trước khi tích hợp.

Giai đoạn 1: Khởi tạo Dự án & Cơ sở Hạ tầng An toàn (Tuần 1 - 4)

Mục tiêu: Thiết lập môi trường phát triển trên NetBeans, xây dựng khung sườn Spring Boot và cơ chế bảo mật cấp cao (Master Key).

Tuần 1: Thiết lập Môi trường NetBeans & Maven

Việc sử dụng NetBeans đòi hỏi cấu hình chuẩn xác để hỗ trợ Spring Boot 3 và Java 17+.

1. Cài đặt Công cụ:

- Tải và cài đặt **Apache NetBeans 20+** (phiên bản hỗ trợ tốt nhất cho Jakarta EE và Spring Boot 3).⁸
- Cấu hình JDK 17 hoặc 21 trong NetBeans (Tools > Java Platforms).

2. Khởi tạo Dự án:

- Sử dụng **Spring Initializr** (start.spring.io) để tạo bộ khung dự án thay vì dùng Wizard cũ của NetBeans để đảm bảo các dependency mới nhất.
- *Cấu hình:* Project: Maven; Language: Java; Spring Boot: 3.2.x; Packaging: Jar.
- *Dependencies:* Spring Web, Spring Data JPA, MySQL Driver, Spring Security, Lombok, Validation.
- **Import vào NetBeans:** Chọn File > Open Project và trở tới file pom.xml. NetBeans sẽ tự động tải các thư viện Maven và xây dựng cây dự án.¹⁰

Tuần 2: Thiết kế Database & Tạo Entity tự động

Tận dụng tính năng Reverse Engineering của NetBeans để tiết kiệm thời gian viết code.

1. Thiết kế Schema: Sử dụng MySQL Workbench để tạo các bảng products, variants, inventory với khóa chính là CHAR(36) cho UUID.

2. Kết nối Database trong NetBeans:

- Mở tab **Services > Databases**. Chuột phải chọn **New Connection**.
- Nhập thông tin kết nối MySQL (Driver: MySQL Connector/J).

3. Generate Entity (Bước quan trọng):

- Trong tab **Projects**, chuột phải vào package com.app.model > **New > Entity Classes from Database**.¹¹
- Chọn kết nối database đã tạo. Chọn tất cả các bảng.
- *Cấu hình:* Bỏ chọn "Generate Named Query Annotations" để giữ code sạch. Đảm bảo chọn "Create Persistence Unit" nếu chưa có.

- **Kết quả:** NetBeans sẽ tự sinh ra các file .java với đầy đủ annotation @Entity, @Id, @Column. Nhiệm vụ của lập trình viên là sửa lại kiểu dữ liệu của ID từ Integer sang String hoặc UUID và thêm annotation @GeneratedValue(strategy = GenerationType.UUID) (đối với Hibernate 6) hoặc tự quản lý việc sinh UUID trong constructor.¹³

Tuần 3: Bảo mật & Master Key

Triển khai yêu cầu về "Khóa mật khẩu riêng cấp cao nhất".¹

1. Cơ chế Master Key:

- Tạo một AuthenticationProvider tùy chỉnh trong Spring Security.
- Logic: Nếu request header chứa X-Master-Key, hệ thống sẽ bỏ qua kiểm tra JWT thông thường và cấp quyền ROLE_SUPER_ADMIN.
- Mã hóa Master Key: Hash chuỗi Master Key bằng BCrypt và lưu trong biến môi trường hoặc file cấu hình đã mã hóa.

2. Mã hóa Cấu hình với Jasypt:

- Thêm dependency jasypt-spring-boot-starter vào pom.xml.¹⁵
- Mã hóa mật khẩu DB và Master Key Hash bằng công cụ dòng lệnh của Jasypt.
- Trong application.properties: spring.datasource.password=ENC(chuỗi_đã_mã_hóa).
- **Cấu hình chạy trong NetBeans:** Chuột phải vào Project > **Properties** > **Run** > **VM Options**. Thêm dòng: -Djasypt.encryptor.password=MẬT_KHẨU_GIẢI_MÃ.¹⁶ Điều này đảm bảo mật khẩu giải mã không bao giờ nằm trong source code.

Tuần 4: Xây dựng API Đồng bộ hóa (Sync API)

1. **Thiết kế DTO (Data Transfer Object):** Tạo các class SyncRequest và SyncResponse để đóng gói dữ liệu delta.
2. **API Endpoint:**
 - POST /sync/push: Nhận danh sách các thay đổi từ client (Insert/Update/Delete). Sử dụng @Transactional để đảm bảo tính toàn vẹn - nếu một bản ghi lỗi, toàn bộ lô đồng bộ sẽ rollback.
 - GET /sync/pull: Nhận tham số lastSyncTime. Sử dụng JPA Specification để truy vấn: SELECT * FROM entity WHERE last_modified > :lastSyncTime.
3. **Unit Test sơ khởi:** Sử dụng JUnit 5 (tích hợp sẵn trong NetBeans) để test logic đồng bộ giả lập.¹⁷

Giai đoạn 2: Quản lý Kho & Tích hợp AI Vision (Tuần 5 - 12)

Mục tiêu: Hiện thực hóa chức năng nhập hàng bằng camera và xử lý ảnh thông minh.

Tuần 5-6: Xử lý Nhập giá Hàng loạt (Batch Processing)

Yêu cầu nhập giá hàng loạt sau khi phân loại.¹

1. **Spring Batch:** Cấu hình một Job trong Spring Batch để xử lý cập nhật giá.
2. **Reader:** Đọc danh sách ID sản phẩm từ API request (ví dụ: tất cả sản phẩm màu "Đỏ", loại "Áo dài").
3. **Processor:** Áp dụng công thức giá (ví dụ: Giá bán = Giá nhập * 1.5).
4. **Writer:** Sử dụng JdbcBatchItemWriter để cập nhật xuống MySQL. Cấu hình `spring.jpa.properties.hibernate.jdbc.batch_size=50` để tối ưu hiệu năng ghi.¹⁸
5. **Giao diện Admin (Giả lập API):** Tạo endpoint cho phép admin gửi tiêu chí lọc và mức giá mới.

Tuần 7-9: Tích hợp YOLOv8 với Deep Java Library (DJL)

Đây là phần lõi của tính năng "nhập hàng bằng camera".¹

1. **Chuẩn bị Mô hình:**
 - Do bạn dùng Java, việc train model trực tiếp bằng Python và export sang định dạng **ONNX** là phương án tối ưu nhất.¹⁹
 - Train YOLOv8 trên tập dữ liệu áo dài/áo bà ba (có thể dùng Google Colab). Export: `yolo export model=best.pt format=onnx`.
2. **Tích hợp vào Spring Boot:**
 - Thêm dependency `ai.djl:api` và `ai.djl.onnxruntime:onnxruntime-engine` vào `pom.xml`.²⁰
 - Tạo Service `ImageRecognitionService.java`.
 - Sử dụng Criteria của DJL để load model ONNX.
 - **Logic Xử lý:**
 - Input: `BufferedImage` từ upload của client.
 - Inference: Chạy model để lấy Bounding Box và Label (Loại áo).
 - Output: Trả về JSON chứa loại áo, độ tin cậy.

Tuần 10: Phân tích Nâng cao - Màu sắc, Họa tiết & Niên đại

1. **Phân tích Màu sắc:** Sau khi YOLO cắt vùng ảnh chứa áo, sử dụng thuật toán **K-Means Clustering** (viết bằng Java thuần hoặc dùng thư viện JavaML) để trích xuất 3 màu chủ đạo (Dominant Colors) và chuyển đổi sang tên màu gần nhất (ví dụ: `#FF0000` -> Đỏ).¹
2. **Dự đoán Niên đại:**
 - Viết hàm logic `ExampleYearPredictor.java`.
 - Lấy `LocalDate.now().getYear()`.
 - Quy tắc nghiệp vụ: Nếu kiểu dáng là "Tay bông" (nhận diện qua YOLO class) -> Năm hiện tại - 1. Nếu "Cổ thuyền" -> Năm hiện tại - 5. (Đây là logic giả định cần được tinh chỉnh theo nghiệp vụ thực tế).
3. **Gắn Metadata:** Sử dụng thư viện **Apache Commons Imaging** để ghi các thông tin này (Mã SP, Năm, Màu) vào metadata (EXIF) của file ảnh gốc trước khi lưu xuống ổ cứng.¹

Tuần 11-12: Quản lý Hình ảnh Đa độ phân giải

1. **Resize Tự động:** Khi ảnh được upload, sử dụng thư viện `Thumbnail` (của Java) để tạo ra 2 phiên bản: `_slide_small` (cho danh sách) và `_slide_large` (cho chi tiết).¹

2. **Tổ chức thư mục:** Lưu ảnh theo cấu trúc `/images/{year}/{category}/{product_id}/` để dễ quản lý backup offline.

Giai đoạn 3: Trải nghiệm Khách hàng & Thuật toán Size (Tuần 13 - 18)

Mục tiêu: Xây dựng tính năng gợi ý đồ và tìm kiếm thông minh.

Tuần 13-14: Thuật toán Fit-Match (Gợi ý Size)

Xây dựng logic "Recommend theo số đo 3 vòng".¹

1. **Cơ sở dữ liệu Size:** Tạo bảng `size_charts` chứa thông số chuẩn (Vòng ngực, eo, hông) cho từng size (S-8X).
2. **Logic Thuật toán (Java Service):**
 - Input: Số đo khách (N, E, M).
 - **Chế độ EXACT:** Tìm size mà $(Size.N \geq Khach.N) \text{ AND } (Size.E \geq Khach.E) \text{ AND } (Size.M \geq Khach.M)$. Sai số cho phép: 2cm.
 - **Chế độ FLEXIBLE (Áo dài):** Ưu tiên Ngực và Eo (trọng số 80%). Vòng hông có thể nới lỏng do tà áo xẻ.
 - **Logic Plus-Size:** Nếu số đo vượt quá bảng chuẩn (8X), hệ thống tự động gợi ý "May đo" hoặc tìm trong kho hàng tồn các mẫu có chất liệu "Co giãn" (Gấm thun).¹

Tuần 15: Tìm kiếm & Gợi ý (Recommender System)

1. **Tìm kiếm:** Sử dụng **Spring Data JPA Specification** để xây dựng bộ lọc động (Dynamic Query) cho phép tìm theo nhiều tiêu chí kết hợp (Màu, Size, Năm, Kiểu) mà không cần viết hàng chục câu query thủ công.
2. **Gợi ý Hàng tồn:**
 - Logic: "Ưu tiên mẫu tồn kho trong năm nay hoặc mới nhập sau ít nhất 4 tuần".¹
 - Query: `SELECT * FROM products WHERE (import_date <= NOW() - INTERVAL 4 WEEK) OR (design_year = YEAR(NOW())) ORDER BY priority_score DESC.`

Tuần 16-18: Giao diện & Trải nghiệm Người dùng (UI/UX Backend Support)

Mặc dù bạn dùng NetBeans xử lý Java, việc thiết kế API phải hỗ trợ UI.

1. **Font & Typography:** Trong API trả về cấu hình (Config API), quy định font chữ là Montserrat cho nội dung và Serif cho tiêu đề nhỏ. Client sẽ đọc cấu hình này để render.
2. **Negative Space:** API phân trang (Pagination) nên trả về số lượng item giới hạn (ví dụ: 10 item/trang) để UI không bị rối, đảm bảo khoảng trắng.

Giai đoạn 4: Thanh toán & Tài chính (Tuần 19 - 24)

Mục tiêu: Tích hợp thanh toán QR và báo cáo doanh thu.

Tuần 19-20: Tích hợp VietQR

1. Tạo Mã QR:

- Sử dụng thư viện Java để tạo chuỗi payload theo chuẩn EMVCo của VietQR.²¹
- Format: 000201010211... chứa BIN ngân hàng, Số tài khoản, Số tiền, và Nội dung (Mã đơn hàng).
- Dùng thư viện ZXing để sinh ảnh QR từ chuỗi payload này.

2. Quy trình: Khi khách chọn "Thanh toán QR", Server sinh ảnh QR, gửi về App. Khách quét và chuyển khoản.

Tuần 21: Webhook & Xử lý Sau thanh toán

1. Webhook: Xây dựng API POST /api/webhook/payment để nhận thông báo từ cổng thanh toán (hoặc ngân hàng) khi tiền về tài khoản.

2. Cập nhật Kho:

- Ngay khi nhận Webhook thành công (xác thực chữ ký số), hệ thống tự động trừ kho: UPDATE inventory SET quantity = quantity - order_qty.
- Nếu giao dịch thất bại, hoàn lại số lượng giữ chỗ (Release Reservation).

Tuần 22-24: Báo cáo & Tài chính

1. Báo cáo Ngày: Sử dụng **Spring Scheduler** (@Scheduled(cron = "0 0 23 * * ?")) để chạy job cuối ngày.

- Tổng hợp: Số lượng bán, Doanh thu, Lợi nhuận (Giá bán - Giá nhập).
- Lưu vào bảng daily_reports để truy xuất nhanh, tránh query lại bảng orders lớn.

2. Bảo mật Giá nhập: Đảm bảo API trả về cho App bán hàng (Client) **không bao giờ** chứa trường import_price. Trường này chỉ hiện trong API dành cho Admin.

Giai đoạn 5: Hệ sinh thái & Marketing (Tuần 25 - 28)

Mục tiêu: Mở rộng tính năng cho thợ sửa và marketing tự động.

Tuần 25: Phân hệ Thợ sửa đồ (Tailor Module)

1. Mô hình Dữ liệu: Thợ sửa là một User có Role = TAILOR.

2. Quy trình:

- Khách tạo yêu cầu sửa -> Hệ thống tạo bản ghi repair_order.
- Thợ nhận đơn -> Cập nhật trạng thái IN_PROGRESS.
- Thanh toán: Tiền được ghi nhận vào ví của thợ (100% revenue). Hệ thống chỉ lưu lịch sử giao dịch.¹

3. Đánh giá: API cho phép khách đánh giá thợ (1-5 sao). Điểm trung bình của thợ được tính lại mỗi khi có đánh giá mới (Event-driven) để tối ưu hiệu năng đọc.

Tuần 26-27: Marketing Tự động (Email & Loyalty)

1. **Tích điểm:** Sử dụng **Spring AOP (Aspect Oriented Programming)**.
 - o Tạo một Aspect chặn phương thức `OrderService.completeOrder()`.
 - o Sau khi đơn hàng hoàn tất, Aspect tự động tính điểm (100k = 1 điểm) và cộng vào tài khoản khách. Cách này tách biệt logic khuyến mãi khỏi logic bán hàng cốt lõi.
2. **Email Nhắc nhở:**
 - o Dùng `JavaMailSender`.
 - o Kịch bản 1: "Abandoned Cart" - Chạy job quét các giỏ hàng không thanh toán sau 24h.
 - o Kịch bản 2: "Review Request" - Gửi email sau 3 ngày nhận hàng, kèm link Google Business Profile.¹

Tuần 28: Kiểm thử Đơn vị & Tích hợp (Unit Testing)

Đây là yêu cầu bắt buộc "Unit test cho từng chức năng trước khi release".¹

1. **Trong NetBeans:** Chuột phải vào Class > **Tools** > **Create/Update Tests**.
2. **Chiến lược Test:**
 - o **Unit Test:** Mockito để giả lập Database, test logic tính giá, logic gợi ý size.
 - o **Integration Test:** Dùng H2 Database (In-memory) để test luồng từ Controller xuống Database.
 - o **Offline Simulation:** Test API đồng bộ với các kịch bản mất mạng giả lập.

Giai đoạn 6: Triển khai & Bàn giao (Tuần 29 - 30)

Tuần 29: Đóng gói & Containerization

1. **Docker:** Tạo Dockerfile để đóng gói ứng dụng Spring Boot. Điều này đảm bảo môi trường chạy trên máy chủ (Production) giống hệt môi trường NetBeans.
2. **Offline Installer:** Đối với yêu cầu cài đặt offline trên PC/Laptop, sử dụng **Launch4j** để gói file JAR thành file .exe có kèm theo JRE (Java Runtime Environment) nhúng sẵn.

Tuần 30: Tài liệu & Chuyển giao

1. **API Documentation:** Sử dụng **Swagger/OpenAPI** (tích hợp sẵn trong Spring Boot) để sinh tài liệu API tự động cho đội phát triển Mobile App.
2. **Hướng dẫn Vận hành:** Viết tài liệu hướng dẫn Admin cách quét mã Master Key, cách sử dụng camera để nhập hàng.

Phần 3: Phụ lục Kỹ thuật & Chi tiết Công cụ (NetBeans Focus)

3.1. Hướng dẫn Thao tác Cụ thể trên NetBeans IDE

Để đạt hiệu quả cao nhất với NetBeans trong dự án này, hãy tuân thủ quy trình sau:

- **Quản lý Maven Dependency:** Đừng sửa file JAR thủ công. Hãy mở file pom.xml trong thư mục Project Files, thêm dependency và nhấn nút **"Reload Changes"** trên thanh công cụ NetBeans để IDE tự động tải thư viện.
- **Debug AI Code:** Khi làm việc với DJL/YOLO, hãy sử dụng tính năng **Debug** của NetBeans (Ctrl+F5). Đặt breakpoint tại dòng xử lý ảnh để xem trực tiếp giá trị mảng pixel, giúp phát hiện lỗi nhận diện sai màu sắc.
- **Database Inspection:** Sử dụng tab **Services > Databases** để truy vấn trực tiếp dữ liệu MySQL ngay trong IDE mà không cần mở công cụ ngoài. Rất hữu ích để kiểm tra dữ liệu sau khi chạy Batch Process.

3.2. Cấu hình Bảo mật Jasypt

Để bảo vệ Master Key và Database Password, không bao giờ lưu text rõ trong code.

Trong NetBeans, khi chạy ứng dụng (Run Project), hãy vào Properties > Run và thiết lập VM Options:

-Djasypt.encryptor.password=YOUR_SECRET_KEY

Điều này đảm bảo rằng ngay cả khi ai đó có được source code, họ cũng không thể giải mã được các cấu hình nhạy cảm nếu không biết YOUR_SECRET_KEY.

3.3. Kết luận

Kế hoạch 30 tuần này không chỉ là một lộ trình phát triển phần mềm mà là một chiến lược chuyển đổi số toàn diện. Bằng cách kết hợp sức mạnh của **Java Spring Boot** (xử lý nghiệp vụ phức tạp), **MySQL** (quản lý dữ liệu lớn), **Offline-First** (ổn định vận hành) và **AI Vision** (tự động hóa), hệ thống sẽ giải quyết triệt để các bài toán thủ công của ngành thời trang truyền thống. Việc tuân thủ nghiêm ngặt quy trình kiểm thử và bảo mật sẽ đảm bảo sản phẩm cuối cùng không chỉ thông minh mà còn an toàn và bền vững.

Nguồn trích dẫn

1. Nghiên cứu xây dựng chức năng ứng dụng.pdf
2. Distributed primary key - UUID, simple auto increment or custom sequential values?, truy cập vào tháng 1 22, 2026, <https://stackoverflow.com/questions/7258619/distributed-primary-key-uuid-simple-auto-increment-or-custom-sequential-value>
3. What is the real advantage of using a UUID instead of an auto-incremented element?, truy cập vào tháng 1 22, 2026, <https://dba.stackexchange.com/questions/322595/what-is-the-real-advantage-of-using-a-uuid-instead-of-an-auto-incremented-element>
4. What are the pros/cons of uuid vs auto-increment serial for the primary key? | OutSystems, truy cập vào tháng 1 22, 2026, <https://www.outsystems.com/forums/discussion/76850/what-are-the-pros-cons-of-uuid-vs-auto-increment-serial-for-the-primary-key/>
5. UUID vs Long primary key - database - Stack Overflow, truy cập vào tháng 1 22,

- 2026, <https://stackoverflow.com/questions/78467932/uuid-vs-long-primary-key>
6. UUID versus Sequence, why is a UUID bad for performance if it is used as index? - Reddit, truy cập vào tháng 1 22, 2026, https://www.reddit.com/r/PostgreSQL/comments/1ckzc8f/uuid_versus_sequence_why_is_a_uuid_bad_for/
 7. Using Delta Sync operations on versioned data sources in AWS AppSync, truy cập vào tháng 1 22, 2026, <https://docs.aws.amazon.com/appsync/latest/devguide/tutorial-delta-sync.html>
 8. NetBeans Platform CRUD Tutorial Using Maven, truy cập vào tháng 1 22, 2026, <https://netbeans.apache.org/tutorial/main/tutorials/nbm-maven-crud/>
 9. Trouble with setting up Java modules in Apache NetBeans 20 Flatpak Installation, on Linux Mint 21.3 Virginia - Stack Overflow, truy cập vào tháng 1 22, 2026, <https://stackoverflow.com/questions/78122310/trouble-with-setting-up-java-modules-in-apache-netbeans-20-flatpak-installation>
 10. Best Practices for Apache Maven in NetBeans, truy cập vào tháng 1 22, 2026, <https://netbeans.apache.org/wiki/main/wiki/MavenBestPractices/>
 11. Making java Entity classes from MySQL tables in netbenas - Stack Overflow, truy cập vào tháng 1 22, 2026, <https://stackoverflow.com/questions/52428654/making-java-entity-classes-from-mysql-tables-in-netbenas>
 12. NetBeans Platform CRUD Application Tutorial, truy cập vào tháng 1 22, 2026, <https://netbeans.apache.org/tutorial/main/tutorials/nbm-crud/>
 13. Using UUID as primary key with Spring Data JDBC | Jonathon Henderson, truy cập vào tháng 1 22, 2026, <https://jonathonhenderson.co.uk/2025/08/03/using-uuid-as-primary-key-with-spring-data-jdbc>
 14. Working with UUIDs in Spring Boot Applications | Medium, truy cập vào tháng 1 22, 2026, <https://medium.com/@AlexanderObregon/working-with-uuids-in-spring-boot-applications-7fb36ee6b598>
 15. ulisesbocchio/jasypt-spring-boot - GitHub, truy cập vào tháng 1 22, 2026, <https://github.com/ulisesbocchio/jasypt-spring-boot>
 16. Encrypted Property Storage in Spring Boot Config Files - Medium, truy cập vào tháng 1 22, 2026, <https://medium.com/@AlexanderObregon/encrypted-property-storage-in-spring-boot-config-files-3ed6374b6f69>
 17. Test Your Spring Boot Applications with JUnit 5 - Okta Developer, truy cập vào tháng 1 22, 2026, <https://developer.okta.com/blog/2019/03/28/test-java-spring-boot-junit5>
 18. Bulk Insert Optimization with Spring Boot and JDBC Batching - Medium, truy cập vào tháng 1 22, 2026, <https://medium.com/@AlexanderObregon/bulk-insert-optimization-with-spring-boot-and-jdbc-batching-57dd031ecad8>
 19. From Python Model to Production: YOLOv8 Drone Detection with Java & Spring Boot, truy cập vào tháng 1 22, 2026,

<https://medium.com/codetodeploy/from-python-model-to-production-yolov8-drone-detection-with-java-spring-boot-346e747b2df9>

20. deepjavalibrary/djl-spring-boot-starter - GitHub, truy cập vào tháng 1 22, 2026,

<https://github.com/deepjavalibrary/djl-spring-boot-starter>

21. thebases/vietqr - GitHub, truy cập vào tháng 1 22, 2026,

<https://github.com/thebases/vietqr>